



M Ű E G Y E T E M 1 7 8 2

Budapesti Műszaki és Gazdaságtudományi Egyetem

Villamosmérnöki és Informatikai Kar

Számítástudományi és Információelméleti Tanszék

Ács Dorottya Julianna

Segédsoftver készítése kódoló algoritmusokhoz

Önálló Laboratórium

KONZULENS

Dr. Pintér Márta

BUDAPEST, 2026

Tartalomjegyzék

Bevezetés.....	3
Motiváció.....	3
Célkitűzések	3
Elméleti háttér	4
Információelméleti alapfogalmak.....	4
Prefix kód	4
Entrópia	4
Átlagos kódszóhossz	5
Optimális kód	5
Statikus Huffman-algoritmus	5
Adaptív Huffman-algoritmus	6
Testvérpár tulajdonság	7
Példa az adaptív Huffman-algoritmus faépítésére.....	8
Rendszertervezés és architektúra	9
Implementáció	10
Statikus Huffman-kódolás megvalósítása	10
Adaptív Huffman-kódolás megvalósítása	11
Gráf megjelenítő implementációja	11
Felhasználói felület.....	12
Statikus Huffman-kódolás felhasználófelülete.....	12
Adaptív Huffman-kódolás felhasználófelülete.....	13
Irodalomjegyzék	14

Bevezetés

Az önálló laboratóriumra választott feladatom az adattömörítő algoritmusok, azon belül is a statikus és adaptív Huffman algoritmusok vizsgálata, és az azok kimenetét és működését demonstráló vizualizációs program fejlesztése volt. Ebben a fejezetben leírom a projektem választása mögötti motivációt, és a megvalósítás során figyelembe vett célokat.

Motiváció

A statikus Huffman-kódolás elméleti alapjaival korábban az informatikai tanulmányaim során, a Kódolástechnika tantárgy keretein belül ismerkedtem meg. Szerettem volna megismerni az adaptív változatát, és jobban belemerülni az adattömörítő algoritmus működésébe.

Célkitűzések

- Algoritmusok megértése és implementálása: A statikus eljárás mellett az adaptív Huffman-kódolás szoftveres megvalósítása, a Vitter-féle algoritmus variációval a gráfban fellépő döntetlenek kezelésére.
- Interaktív vizualizáció megvalósítása: Egy olyan grafikus felület létrehozása, ami az adaptív (on-line) algoritmust lépésenként bemutatja, ezzel jobban demonstrálja a karakterenkénti kódolás működését.
- Technológiai fejlődés és webes környezet: Céлом volt a kliensoldali webes technológiák mélyebb megismerése, és a feladathoz látványos és platformfüggetlen grafikus megjelenítésre volt szükség. Ezért fejlesztéshez a JavaScript nyelvet választottam, a gráfok rendereléséhez pedig a D3.js adatvizualizációs könyvtárat használtam.
- Felhasználóbarát architektúra: Egy letisztult, böngészőből azonnal futtatható, egyoldalas webalkalmazás kialakítása, amely nem igényel szerveroldali függőségeket vagy bonyolult telepítési folyamatokat, amit később akár fel is tehetek saját domain-re.

Elméleti háttér

Ebben a fejezetben leírom a projekt során megvalósított statikus és adaptív Huffman-algoritmusok információelméleti alapjait, legfontosabb tulajdonságait és működési elvét.

Információelméleti alapfogalmak

Amikor egy szoftver adatokat küld hálózaton, vagy fájlokat ment a háttértárra, a célunk az, hogy ezeket az információkat minél kevesebb bittel írjuk le a felhasznált sávszélesség vagy tárhely minimalizálására. A veszteségmentes adattömörítés garantálja, hogy a tömörített, rövidebb adatból az eredeti tartalom bitre pontosan visszaállítható legyen.

Prefix kód

Egy kód akkor prefix, ha a lehetséges kódszavai közül egyik sem folytatása a másiknak, vagyis bármelyik kódszavának végéből bármekkora részletet levágva nem kapunk egy másik kódszót. Minden kódolás ábrázolható kódfával, azonban a nem prefix kódoknál a fa belső csúcsaihoz is tartozhat kódszó. Ezzel szemben a prefix kódok kódfájában a kódszavak kizárólag a levelekben helyezkednek el, ami garantálja, hogy egyik kódszó sem leszarmazottja a másiknak. Ennek köszönhetően a gyökértől a levelekig vezető utak egyértelműen és dekódolhatóan jelentik a kódszavakat.

Entrópia

A diszkrét valószínűségi változó entrópiája az információ bizonytalanságát fejezi ki. Megadja azt az alsó határt, hogy átlagosan minimum hány bitre van szükség egy karakter kódolásához az adott szövegben. Az X valószínűségi változó entrópiájának képlete:

$$H(X) = - \sum_{i=1}^n p(x_i) \log p(x_i)$$

ahol $p(x_i)$ az x_i karakter előfordulási valószínűsége.

Átlagos kódszóhossz

Az átlagos kódszóhossz a forrás elemeihez rendelt kódszavak hosszának várható értéke. Ennek a minimalizálása a tömörítő algoritmusok célja. Az átlagos kódszóhossz képlete:

$$L = \sum_{i=1}^n p(x_i) |f(x_i)|$$

ahol $p(x_i)$ az x_i karakter előfordulási valószínűsége, és $|f(x_i)|$ az x_i -hez rendelt kódszó hossza.

A Shannon-tétel szerint az átlagos kódszóhossz nem lehet kisebb az entrópiánál, az adattömörítés célja tehát az entrópia minél jobb megközelítése.

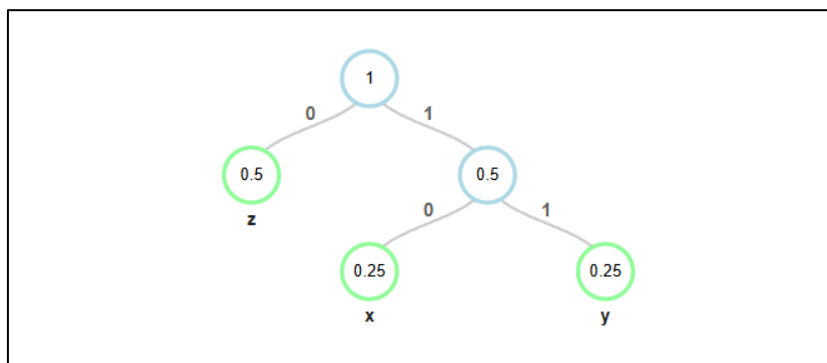
Optimális kód

Akkor nevezünk egy veszteségmentes kódolást optimálisnak, ha az átlagos kódszóhossza minimális. Vagyis, az adott bemenetre nem lehet másik kódolást találni, ami hibátlanul dekódolható, és rövidebb az átlagos kódszóhossza.

Statikus Huffman-algoritmus

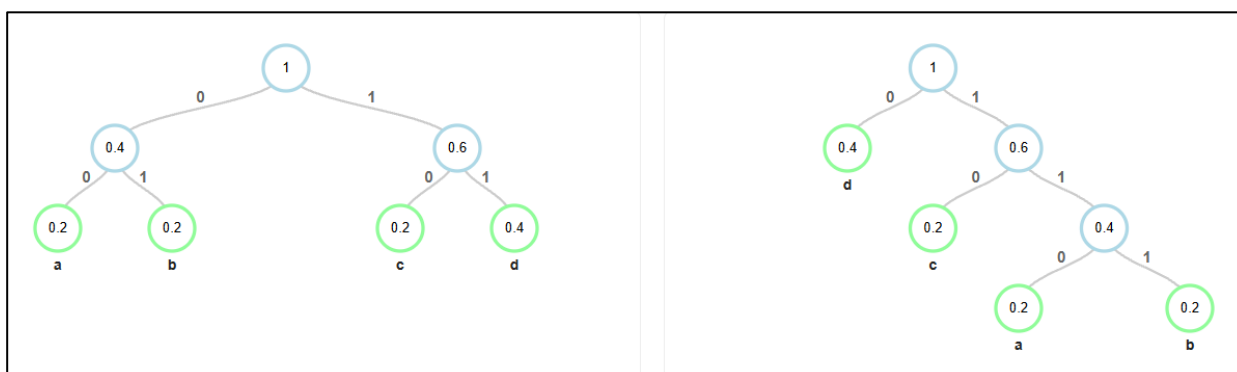
A statikus Huffman-kódolás optimális, prefix kódolás. Gyakoriságfüggő, a forráseloszlás ismeretében lehet csak alkalmazni. Ha a forráseloszlás ismeretlen, relatív gyakorisággal becsülhető a karakterek előfordulási valószínűsége, de ehhez előre végig kell olvasni a bemenetet. A Huffman-fa felépítésének lépései:

- 1) Minden szimbólumnak megfeleltetünk egy levelet
- 2) Kiválasztjuk a két legkisebb valószínűségű különálló levelet: x, y
- 3) Összevonjuk őket egy csomópontba, aminek a valószínűsége $p(x) + p(y)$
- 4) Megcímkézzük az új éleket 0-ra és 1-re
- 5) Ha a gráf nem összefüggő, ugrás a 2) lépésre



1. ábra: Példa egy egyszerű Huffman-fára ('xyzz' bemenet)

Az algoritmus eredménye több azonos valószínűség esetén, vagy a nullák és egyesek felcserélésével nem egyértelmű. A különböző kódok alakjukban is eltérhetnek, de az átlagos kódszóhossz mindig azonos lesz.



2. ábra: Példa egy nem egyértelmű kódolásra ('abcd' bemenet)

Adaptív Huffman-algoritmus

Az adaptív Huffman-kódolás azt a problémát oldja meg, ha olyan adatfolyamot szeretnénk tömörítve kódolni, aminek nem ismerjük előre az eloszlását, és a szöveget nem szeretnénk vagy nem tudjuk előre végigolvasni az eloszlás becsléséhez. Az on-line kódolás során a kódolást mindig a pillanatnyi becsült eloszlás szerint végezzük. Több megvalósítása is létezik ennek a kódolásnak, én a Vitter-algoritmust használtam.

A gyakorlati implementáció során célszerű a relatív gyakoriságok lépésenkénti újraszámítása helyett a karakterek előfordulási számát használni. Ez az egyszerűsítés azért alkalmazható, mert a darabszámok nagyság szerinti sorrendje megegyezik a relatív gyakoriságokéval (mivel a relatív gyakorisághoz az előfordulási számot osztanánk az eddigi összkarakter számmal, ami egy azonos, konstans osztást jelentene minden karakterre). Ezért a megvalósításnál a csomópontok súlyai egész számok maradnak, és nem kell minden új karakter

bevitelekor végigfrissíteni a fa minden súlyát, elég csak a módosított levéltől a gyökérig vezető úton frissíteni minden csúcsot.

Testvérpár tulajdonság

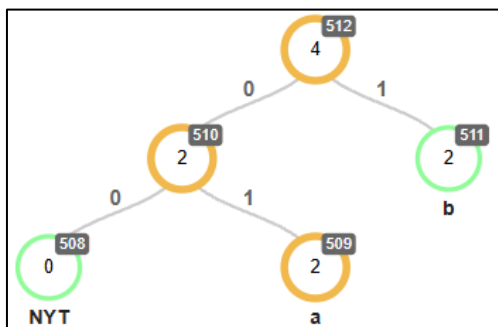
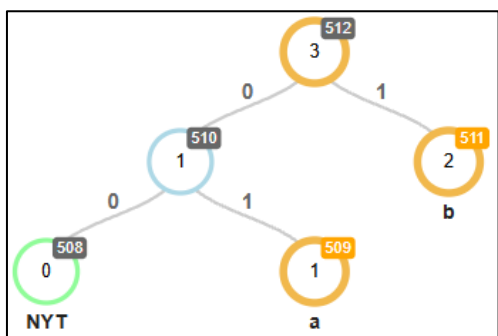
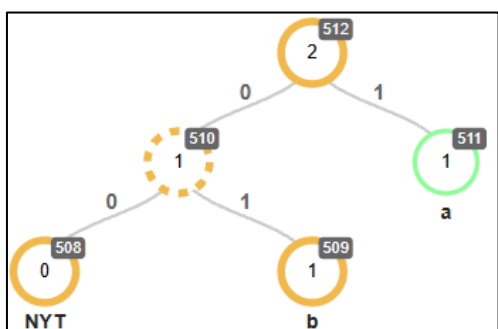
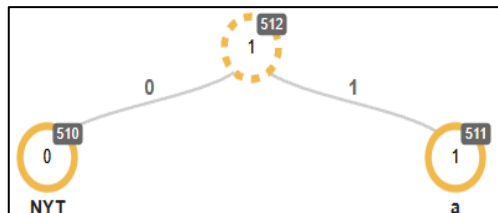
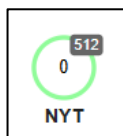
A testvérpár tulajdonság akkor igaz egy fára, ha a gyökértől a levelek felé haladva felsorolhatóak a fa testvér pontpárjai súlyuk (előfordulási valószínűségük) szerint nem növekvő sorrendben (vagy gyakoriság szerinti súlyozásnál nem csökkenő sorrendben). Ez minden Huffman-fára igaz. Az egyező súlyú csúcsok tehát mindig egymás mellett vannak ebben a felsorolásban, ezt egy blokknak hívjuk.

A kódolás során használunk egy NYT (Not Yet Transmitted) csúcsot, ami egy nulla súlyú csúcs, ezt még a kódolás megkezdése előtt felvesszük a gráfba. Emellett a csúcsokat a kódolás során folyamatosan sorszámozzuk annak érdekében, hogy egyértelmű legyen a kódolás kimenete. A kódolás során minden karakter beolvasása után a fát úgy kell frissíteni, hogy ne sérüljön a testvérpár tulajdonság, és minden szülő súlya megegyezzen a gyerekeinek súlyával. A kódolás és karbantartás lépései:

- 1) Az érkező karakter vizsgálata, és kódolása. A kódoló kapott karaktertől függően két dolgot tehet:
 - Ismert karaktert kapott: Ekkor a kódoló továbbítja a fában már korábban felvett levélhez tartozó kódszót (gyökértől levélig élenkénti bitek).
 - Még nem látott karaktert kapott: Ilyenkor a kódoló kiküldi az NYT csúcs kódszavát (ezzel jelzi a dekódolónak, hogy új karaktert kapott), utána kiküldi a kapott karaktert még nem kódolt alakban (pl. az ASCII kódját). Ezután létrehoz az új karakternek egy levelet úgy, hogy az NYT csomóponthoz rendel két gyereket. A bal oldali gyereke lesz az új NYT csúcs, a jobb pedig az új karakter.
- 2) Vizsgált csúcs = érkező karakter
- 3) Kicseréljük a vizsgált csúcsot a blokkjában lévő legnagyobb sorszámú csúccsal (a hozzá tartozó részfával együtt mozgatva)
- 4) Növeljük a csúcs súlyát eggyel
- 5) A vizsgált csúcs szülője lesz az új vizsgált csúcs, ha az nem a gyökér, ugrás a 3) lépésre

Példa az adaptív Huffman-algoritmus faépítésére

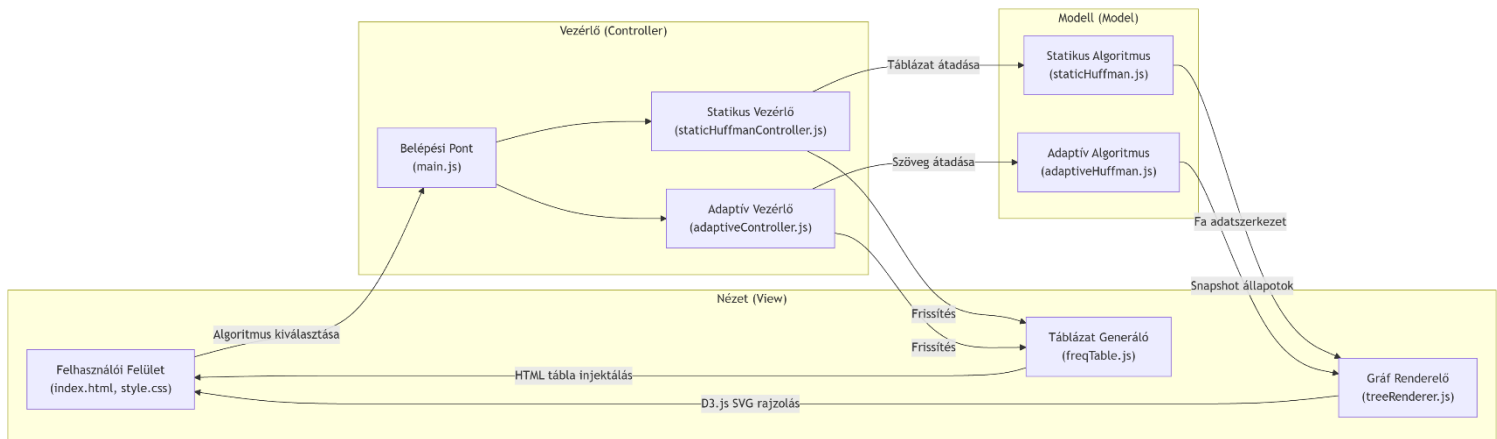
Az 'abba' karaktorsor kódolása:



1. A kódolás legelején még csak az NYT csúcsból áll a gráf. Ez a gyökér, és az 512-es számozást kapja. Ez a kezdőszám garantálja, hogy minden szimbólumnak biztosan jusson levél, mert az ASCII karakterábécé 256 karaktert tartalmaz, és mivel egy N levelű bináris fának legfeljebb $2N-1$ csomópontja lehet, tehát az ASCII szimbólumokból felépített fának maximum 511 számot kell kiosztani.
2. Az első karakter ('a') beolvasása után a gyökérnek két gyerek lett, amik közül a bal oldali az NYT, és a jobb oldali az új levél 1-es súllyal, és a gyökér súlya is megnő 1-re.
3. A második karakter beolvasásakor az NYT pont megint szülő lesz, és a gyerekei az új NYT és az új 'b' levél, és a hozzáadás után a fa súlyai is frissülnek.
4. A harmadik karakter 'b', helyet cserél a blokkjában legnagyobb számú levéllel, az 'a'-val, és a 'b' levél és a gyökér értéke is nő eggyel.
5. Egy újabb 'a' hozzáadásakor csak a már meglévő levél súlyát kell frissíteni, viszont ez végig frissül a gyökérig, így 3 csúcs értéke is nő.

3. ábra: Az 'abba' karaktorsor Adaptív Huffman-kódolásának lépései

Rendszertervezés és architektúra



4. ábra: A szoftver architektúra diagramja

A program fejlesztése során az egyik legfontosabb szempont a modularitás, a funkciók logikai szétválasztása volt. A kód az MVC (Modell-Nézet-Vezérlő) tervezési minta alapelveit követi, amely biztosítja, hogy az üzleti logika és a grafikus megjelenítés teljesen függetlenek legyenek egymástól, így később újabb algoritmusok hozzáadása az architektúra felépítésének változtatása nélkül elvégezhető.

A rendszer három fő rétegre bontható:

- **Vezérlők (Controllers):** A main.js felel a belépési pontért és az algoritmusok közötti váltásért. A tényleges eseménykezelést (gombnyomások, léptetés, felhasználói adatbevitel) a staticHuffmanController.js és az adaptiveController.js modulok végzik.
- **Modellek (Encoders):** Ide tartoznak a tiszta algoritmikus logikát megvalósító fájlok (staticHuffman.js, adaptiveHuffman.js). Ezek függetlenek a grafikus felülettől, és kizárólag a bináris fák adatszerkezetének építésével és karbantartásával foglalkoznak.
- **Nézet (View):** A grafikus kimenet generálásáért felelős réteg. A treeRenderer.js a D3.js könyvtár segítségével alakítja át a modellek által felépített adatstruktúrát SVG vizualizációvá, míg a freqTable.js a dinamikus gyakorisági táblázatok HTML renderelését végzi.

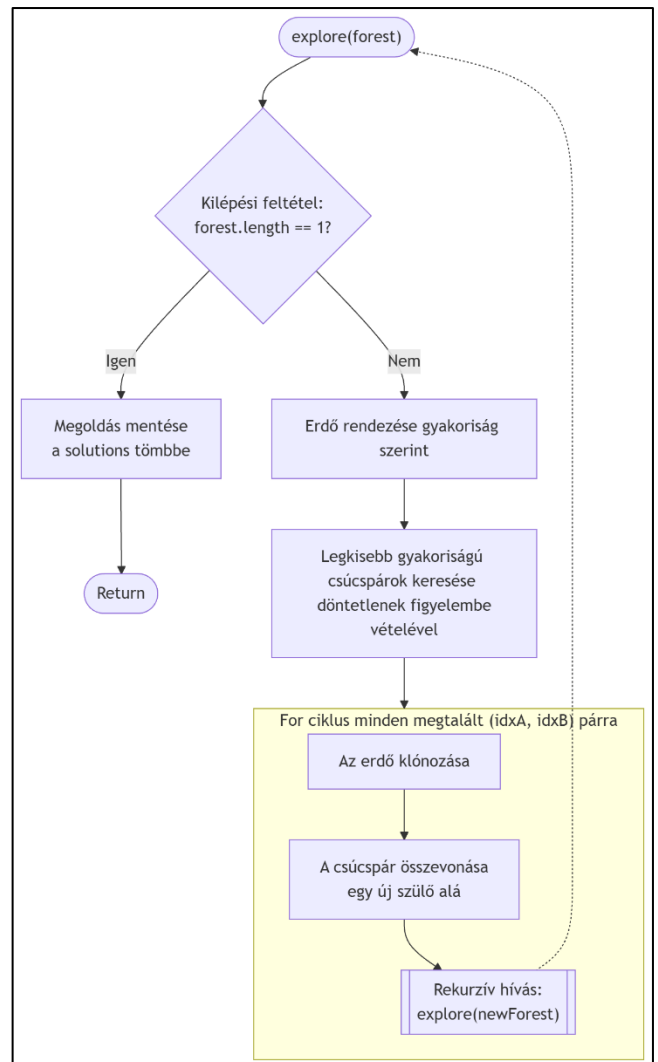
Implementáció

A szoftver implementációja a félév során egy iteratív folyamatként valósult meg, szoros együttműködésben a témavezetővel. Mivel az alkalmazás célja a vizuális szemléltetés, a kódolás a grafikus felület és a D3.js alapú gráfmegjelenítő kialakításával kezdődött. Ez biztosította, hogy az algoritmusok kimenete a későbbi fázisokban azonnal, vizuálisan is tesztelhető legyen. A megjelenítési keretrendszer elkészítése után az algoritmusok megvalósítása következett. Elsőként a statikus Huffman-kódolás rekurzív algoritmusát implementáltam, majd annak sikeres tesztelése után készült el a futásidőben frissülő adaptív (Vitter-féle) megoldás. A fejlesztést végigkísérő rendszeres konzultációk garantálták, hogy az algoritmusok helyesen működnek, és kielégítik a felhasználói elvárásokat.

Statikus Huffman-kódolás megvalósítása

A statikus Huffman-kódolás faépítését egy visszalépéses rekurzív algoritmus végzi, amelyet az alábbi folyamatra szemléltet. A rekurzív függvény első meghíváskor a különálló karakterlevelekből álló erdőt kapja, majd a rekurzív hívásokat addig végzi, amíg egy egybefüggő gráfot kap.

A hagyományos implementációkkal ellentétben ez a megvalósítás a döntetlen helyzeteknél az összes lehetséges fa-alternatívát feltérképezi. Ezt úgy teszi, hogy a legkisebb gyakoriságú csúcsok párosításakor minden lehetséges döntetlen kombinációra lemásolja az addigi félkész kódfát, és párválasztásonként különbözően építi utána végig. Az algoritmus megtalálja az összes lehetséges kimenetet és eltárolja, ez biztosítja később, hogy megjeleníthető legyen mindegyik lehetséges kódfa.



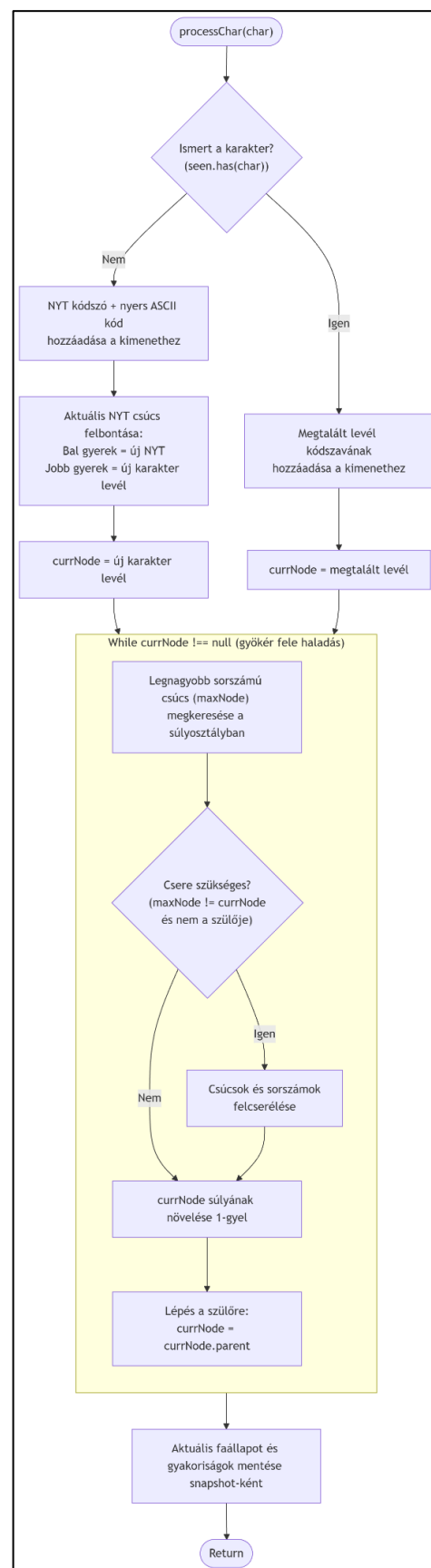
5. ábra: A statikus Huffman-kódoló függvény rekurzív folyamata

Adaptív Huffman-kódolás megvalósítása

Az adaptív Huffman-kódolást a korábbi fejezetben elméleti szinten már ismertetett Vitter-algoritmus alapján valósítottam meg szoftveresen, ennek a karakterfeldolgozó és állapotmentő folyamatát az alábbi ábra szemlélteti. A *processChar* függvény a bemeneten érkező karaktert dolgozza fel: új elem beérkezésekor az aktuális NYT csúcsot bontja fel egy új NYT csúcsra és egy karakterlevélre, míg ismert karakter esetén közvetlenül a gráfban már meglévő levélről indul. A fa futásidőben történő átstrukturálását a testvérpár-tulajdonság ellenőrzése és fenntartása irányítja, a vizsgált levélről egészen a gyökérig haladva. Ezt úgy teszi, hogy amennyiben egy csúcs nem a legnagyobb sorszámu a saját súlyosztályában, az algoritmus a súly növelése és a továbblépés előtt felcseréli azt a megfelelő elemmel, így a növelés után az új súlyosztályában az utolsó helyen lesz. A folyamat végén állapotmentést készít (snapshotot), hogy a kódolás folyamatát végig lehessen követni.

Gráf megjelenítő implementációja

A szoftver grafikus kimenetért a *treeRenderer.js* modul felel, amely a *D3.js* könyvtárra épül. A vezérlőtől kapott absztrakt fagráfot a *D3.js* beépített hierarchia- és fa-elrendezési (*d3.tree*) funkcióival alakítja át szabványos SVG elemekké. A program dinamikusan számítja ki a rajzvásznon magasságát a fa mélysége alapján, megelőzve ezzel a szintek torzulását és egymásra csúszását. Adatkötés segítségével a csomópontokhoz köröket és feliratokat, az élekhez pedig vonalakat rendel, automatikusan ráillesztve a karaktereket, a gyakoriságokat és az élek 0 vagy 1 bitértékeit. Az adaptív algoritmus lépésenkénti vizualizációja során a snapshotok közti léptetésekkor letisztítja és újrarendeli a gráfot. Az algoritmus követését segíti, hogy a végbement változásokat kiemeli színezéssel a megjelenített fagráfon.



6. ábra: Az Adaptív Huffman-kódoló függvény lépései egy beérkező karakterkor

Felhasználói felület

A felhasználói felületet igyekeztem letisztultnak és intuitívnak tervezni. A program külön oldalban jeleníti meg az algoritmusokat, ezek között egy legördülő menüben lehet váltani.

Statikus Huffman-kódolás felhasználófelülete

Coding Visualizer

Algorithm: Static Huffman

Static Huffman Coding

retékGenerateShow All Variations

Character	Frequency	
r	0.2	X
t	0.2	X
k	0.2	X
e	0.4	X

+ Add Row

7. ábra: A kódoló program Statikus Huffman-kódoló oldala ('reték' bemenettel)

A statikus Huffman-kódolás bemenetét kétféleképpen lehet bevinni: szöveg beírásával az oldal szövegmezéjébe, vagy eloszlás beírásával az eloszlástáblázatba (az **+ Add Row** gomb használatával tetszőleges mennyiségű sor, vagyis karakter bevihető, és az **X** gombbal törölhető az adott sor). Hibás eloszlás bevitelekor a program egy hibaüzenetet jelenít meg a felhasználónak.

A bemenet megadása után a **Generate** gombbal lehet a kódolást elvégezni.

A **Generate** gomb mellett a **Show All Variations** jelölőnégyzettel be- és kikapcsolható az összes megoldás megjelenítése, kiszűrhetőek azok a gráfok, amik alakban nem különböznek.

Frequencies must sum to 1 (Current: 1.1000)

8. ábra: Hibás bemenet esetén a hibaüzenet kiírja a megadott karakterek valószínűségeinek összegét

Adaptív Huffman-kódolás felhasználofelülete

Coding Visualizer
Algorithm: Adaptive Huffman ▾

Adaptive Huffman Coding

reték Start Reset Append... Append

<< < Step: 4 / 5 > >> Char: e | Processed Text: reték | Encoded Binary: r0e00t11

Char	Weight
e	2
r	1
t	1

```
graph TD; 512((512  
4)) -- 0 --> 510((510  
2)) --- e[e]; 512 -- 1 --> 511((511  
2)); 511 -- 0 --> 508((508  
1)); 511 -- 1 --> 509((509  
1)) --- r[r]; 508 -- 0 --> 506((506  
0)) --- NYT[NYT]; 508 -- 1 --> 507((507  
1)) --- t[t];
```

9. ábra: A kódoló program Adaptív Huffman-kódoló oldala ('reték' bemenettel, a második 'e' lépésnél)

Az adaptív algoritmust a **Start** gomb lenyomásával lehet megkezdeni. Ennek hatására a program beolvassa a szöveget, és megjeleníti az első lépést.

Az oldal minden lépésnél megjeleníti a jelenleg kódolt karaktert (az ábrán 'e'), az eddig feldolgozott karakterláncot (az ábrán 'rete'), a kimeneten lévő karaktersort (az ábrán 'r0e00t11') és az eddig kódolt karakterek előfordulásainak számát táblázatos formában.

A programot léptetni a **<** **>** gombokkal lehet, és az első és utolsó lépésekre ugráshoz a **<<** **>>** gombokat lehet használni.

Mivel ez egy on-line algoritmus, a bevitt szöveg végére új karakterek fűzésével a korábbi kimenet nem változik. Az **Append...** szövegdobozba bevitt szöveget az **Append** gomb megnyomásával lehet hozzáfűzni a bemenethez.

A **Reset** gombbal lehet kiüríteni a program bemenetét, a megjelenítő visszaáll alapállapotba (üres bemenet, nincs gráf kirajzolva).

A szoftver elérhetősége

Az teljes projekt, a forráskódok, valamint a webalkalmazás verziókövetési előzményei elérhetők és megtekinthetők a [GitHub repository-jában](#).

Irodalomjegyzék

- [1] Györfi László, Györi Sándor, Vajda István: Információ- és kódelmélet
- [2] Pintér Márta: Információelmélet – Huffman-kód PPT
- [3] Buttyán Levente, Györfi Sándor, Vajda István: Kódolástechnika
- [4] Adaptív Huffman-kódolás angol Wikipédia oldala:
https://en.wikipedia.org/wiki/Adaptive_Huffman_coding
- [5] Adaptív Huffman-kódolást magyarázó angol nyelvű videó:
https://www.youtube.com/watch?v=1D0RbQw9_6o&t=56s
- [6] Fagráfok JavaScript-beli programozását leíró oldal:
<https://www.geeksforgeeks.org/javascript/tree-in-javascript/>
- [7] D3.js megjelenítő JavaScript könyvtár fa osztályának oldala:
<https://d3js.org/d3-hierarchy/tree#tree>
- [8] A projekt GitHub repository-ja:
<https://github.com/Dohogo1/Onlab>